

secure digital document vault

final presentation · equipo 7

Barrios · Caballero · Contreras · Martínez

Mayo 2026

Criptografía · Dra. Rocío Aldeco Pérez · UNAM 2026-2

1 · System Overview

**Hoy enviamos archivos
sensibles
por canales que no
garantizan nada.**

Correo

Sin confidencialidad ni

**Drive /
Dropbox**

WhatsApp

Cifrado limitado, sin

**La seguridad
no se confía,
se demuestra.**

Con criptografía formal, no con promesas.

¿Qué construimos?

SDDV — Secure Digital Document Vault

CLI en Python para almacenar y compartir archivos de forma segura

300

tests verdes

32

commits

7

vulns resueltas

D2 · AEAD

D3 · Híbrido

D5 · Firma

D6 · Keystore

Arquitectura por capas

Aplicación · CLI python -m crypto

Composición · `secure_send.py`

confiable

Firma · Ed25519 sobre SDDH

Híbrido · X25519 KEM + AEAD DEM

Simétrico · AES-256-GCM / ChaCha20-Poly1305

Llaves · `script` + AES-GCM

confiable

2 · Threat Model

Activos a proteger

Contenido del archivo

Información sensible → AEAD (D2)

Passwords

Solo en memoria · longitud mínima 12

Metadatos

Filename, timestamp, destinatarios
→ AAD del DEM

Firmas digitales

No-repudio → Ed25519 (RFC 8032)

Llaves privadas

Ed25519 + X25519 → scrypt + AES-
GCM (D6)

Nonces AEAD

Reuso = catastrófico → CSPRNG
fresco

Adversarios considerados

ADV-1 Externo

Lee y modifica contenedores

ADV-4 Acceso físico

Copia el keystore

ADV-2 Destinatario malicioso

Lee su archivo legítimamente

ADV-5 Fuerza bruta offline

Prueba passwords

ADV-3 Man-in-the-Middle

Sustituye pubkeys

ADV-6 Dispositivo comprometido

Keylogger, RAM dump → fuera de scope

Lo que SDDV asume del entorno

- CSPRNG del SO es seguro
- Pubkeys distribuidas auténticamente
- Usuario elige passwords fuertes
- Sin malware en ejecución

3 · D2 — AEAD

Selección del algoritmo

AES-256-GCM

Default · NIST SP 800-38D

- ❑ Acelerado por hardware (AES-NI)
- ❑ Estándar de la industria
- ❑ TLS 1.3, IPsec

ChaCha20-Poly1305

RFC 7539

- ❑ Sin instrucciones AES
- ❑ Constant-time por diseño
- ❑ Ideal para móviles

¿Por qué AEAD y no «encrypt + MAC»?

Un solo mecanismo garantiza confidencialidad e integridad. Diseñar a mano es donde surgen POODLE, Lucky13, BEAST.

El AAD no es opcional

MAGIC(4) "SDDV"	
VERSION(1) = 1	
ALGO_ID(1)	
TIMESTAMP(8) Unix UTC big-endian	
FNAME_LEN(2) uint16	
FILENAME UTF-8	
NONCE(12) · CT_LEN(4) · CIPHERTEXT	
TAG(16)	

← Cabecera completa pasa como AAD al DEM

Propiedad clave: modificar 1 bit del filename, timestamp o algoritmo ⇒

InvalidTag al descifrar

96

bits aleatorios · CSPRNG · fresco por archivo

Nonce reuse en GCM es CATASTRÓFICO

Expone la authentication key.

Cada archivo usa clave fresca $\Rightarrow$ el límite es por-clave, no global.

3 · D3 — Cifrado Híbrido

Una llave **efímera** por
destinatario.
Un archivo **cifrado una
vez.**

KEM (Key Encapsulation) — X25519 ECDH:

- Genera `file_key` aleatoria de 256 bits
- Por cada destinatario: par X25519 efímero $\rightarrow$ ECDH $\rightarrow$ HKDF

`\wrap_file_key`

Las llaves efímeras se destruyen tras el wrap.

Si en el futuro se filtra una `recipient_priv`, los mensajes pasados **quedan protegidos** porque las efímeras ya no existen.

Contenedor SDDH

```
MAGIC(4) "SDDH" · VERSION · ALGO · TIMESTAMP(8)
FNAME_LEN(2) · FILENAME · RCPT_COUNT(2)
```

```

┌─ Por cada destinatario (124 B) ─┐
│ FINGERPRINT(32)  SHA-256(raw X25519 pubkey)
│ EPH_PUB(32)      clave efímera
│ WRAP_NONCE(12)   nonce del wrap
│ WRAPPED_KEY(48)  file_key cifrada (32 ct + 16 tag)
└──────────────────────────────────┘

```

← fin del AAD del DEM

NONCE(12) · CT_LEN(4) · CIPHERTEXT · TAG(16)

3 · D5 — Firma Digital

¿Por qué Ed25519?

Propiedades

- ❑ Curve25519 · 128 bits de seguridad
- ❑ Llaves de 32 B
- ❑ Firma de 64 B
- ❑ Constant-time por diseño

Determinismo

No requiere CSPRNG en cada firma.

Inmune al ataque PS3 (Sony 2010): reuso de nonce en ECDSA expuso la llave maestra.

$$\text{SIGN_MAGIC}(4) + \text{SIGNER_FP}(32) + \text{SIGNATURE}(64) = 100 \text{ B}$$

Verify-first

Patrón ingenuo

decrypt $\rightarrow$ verify

expone plaintext si firma falla

posible timing oracle

vs.

API SDDV

verify $\rightarrow$ **decrypt**

no toca el descifrado

tiempo uniforme

```
def secure_verify_and_decrypt(signed_container, expected_signer_pub,
                             recipient_priv, max_age_seconds):
    container = verify_hybrid_container(    # [] VERIFY
        signed_container, expected_signer_pub)
                                           # InvalidSignature  $\rightarrow$  ST
    return decrypt_for_recipient(          # [] DECRYPT (solo si ver
        container, recipient_priv, max_age_seconds)
```

El fingerprint del firmante está **dentro** de lo firmado.

Un atacante **no puede sustituir** el firmante por Eve manteniendo la firma de Alice.

`SIGNATURE = Ed25519(SDDH || "SIGS" || SIGNER_FP)`

3 · D6 — Key Management

scrypt + AES-256-GCM

KDF — scrypt RFC 7914

Memory-hard → penaliza GPU/ASIC

n $2^{15} = 32\,768$

r 8

p 1

dklen 32 B

salt 16 B CSPRNG

Costo deliberado

150 ms y 80 MiB RAM
por intento en laptop

Con password de 50 bits + 1000 GPUs:
≈ 2 700 años

Keystore JSON

```
{  
  "version": 1, "name": "alice", "status": "active",  
  "kdf": {  
    "algorithm": "scrypt", "salt_b64": "...",  
    "n": 32768, "r": 8, "p": 1, "dklen": 32  
  },  
  "encryption": {  
    "algorithm": "AES-256-GCM",  
    "nonce_b64": "...", "tag_b64": "..."  
  },  
  "encrypted_private_key": "...",  
  "public_keys": {"ed25519_pub_b64": "...", "x25519_pub_b64": "..."},  
  "fingerprints": {"ed25519": "...", "x25519": "..."},  
  "metadata": {"expires_at": null, "rotated_from": null}
```

Ciclo de vida

init

unlock

change-pwd

rotate

revoke

delete

backup

restore

Política de no-caching: cada unlock re-deriva con script.

La llave privada vive solo en el frame del caller.

4 · Secure Practices

Las 4 prácticas que aplicamos

Input validation

- `validate_filename` (CWE-22)
- `validate_timestamp` (CWE-294)
- `ciphertext_length` 100 MiB (CWE-770)
- `MAX_RECIPIENTS` = 1024

Fail-closed

- Verify antes de decrypt
- `unlock` bloquea si `revoked/expired`
- AEAD nunca devuelve plaintext si tag falla

Canonicalization

- `struct.pack(">Q",...)` big-endian
- JSON con `separators=(",",":")`
- `posixpath.normpath` check

Error handling

- `InvalidTag` · `InvalidSignature`
- `IdentityRevokedError`
- Sin atrapar excepciones genéricas

**Si la API es difícil
de usar mal,**

**los errores de
implementación**

**se reducen
drásticamente.**

5 · Security Audit

Vulnerabilidades resueltas

VULN-001 · ALTA

Path traversal en filename

CWE-22 · Fix: validate_filename

VULN-005

Type confusion SDDH→SDDV

Fix: detección de magic

VULN-002

Replay sin freshness check

CWE-294 · Fix: validate_timestamp

VULN-006

Path traversal al unpacking

CWE-22 · Fix: safe_path_join

VULN-003

DoS por ct_len sin tope

CWE-770 · Fix: 100 MiB cap

VULN-007

Password débil en PKCS8

CWE-521 · Fix: MIN_LEN=12

Lo que NO protegemos

✗ Dispositivo comprometido

Keylogger, RAM dump □ requiere HSM

✗ Side-channel físico

Cold boot, EMI □ hardware

✗ Distribución de pubkeys

No hay PKI; canal fuera de scope

✗ Pérdida de pwd + backup

Propiedad criptográfica, no bug

✗ Revocación distribuida

Sin CRL/OCSP; revoke local

✗ Quantum adversary

Roadmap: hybrid PQ (RFC 9180)

Documentar lo que NO se hace es tan importante como documentar lo que sí.

6 · Frontend Web

Todo corre en el navegador. Cero backend.

Stack

- **Pyodide v0.27.2** — CPython en WASM
- cryptography (wheel oficial)

Zero-divergence

El workflow Pages hace:

```
cp -r crypto _site/crypto
```

El frontend y el CLI ejecutan los **mismos**

Defensas del frontend

Lo que mantiene

- ❑ Verify-first del D5
- ❑ AAD del DEM (D2/D3)
- ❑ Fail-closed: InvalidTag/InvalidSignature
- ❑ Llaves privadas **nunca** dejan el navegador
- ❑ Sin telemetría, sin terceros para datos

Nuevos riesgos

- × **ADV-3** (supply chain): mitigado con **SRI sha384** sobre Pyodide
- × **ADV-6** amplificado: navegador = más superficie. Recomendamos CLI para secretos sensibles.
- × XSS / clickjacking: mitigado con **CSP estricta** + frame-ancestors 'none'

7 · Final Demo

Lo que vamos a mostrar

1. Init identidades **alice + bob**
2. Alice cifra y firma para Bob
3. Bob verify + decrypt → **plaintext**
4. Password incorrecto → **InvalidTag**
5. Contenedor modificado → **InvalidSignature**
6. Rotate keys de Alice
7. Backup → delete → restore

```
python demo_d6.py  
bash verificar_d6.sh
```

Estado final

300 /
300

tests verdes

4500

4

7

32

Referencias

Estándares y guías

Estándares · RFCs

- NIST SP 800-38D — GCM
- RFC 7539 — ChaCha20 + Poly1305
- RFC 7748 — X25519 (Curve25519)
- RFC 7914 — scrypt KDF
- RFC 8032 — Ed25519 (EdDSA)
- RFC 9180 — HPKE
- RFC 5869 — HKDF

Guías · Librerías

- OWASP Password Storage Cheat Sheet
- OWASP Cryptographic Storage
- libsodium / NaCl misuse-resistant
- cryptography (pyca) — primitivas
- hashlib.scrypt — stdlib
- pytest — testing

¿Preguntas?

Equipo 7

Barrios · Caballero · Contreras · Martínez

github.com/milliyx/Cryptography